JAiO - praca domowa nr 3

Tymoteusz Bultrowicz

May 2026

1 Definicja:

Oznaczamy **automat z klawiaturą** $\mathcal{K}$ jako krotkę $(A, Q, q_0, q_{tak}, T, \sigma, \delta)$. Gdzie:

A - alfabet wejściowy

Q - zbiór stanów automatu

q_0 - stan początkowy automatu

q_{tak} - stan akceptujący automatu

T - alfabet taśmowy, $A \subset T$

$\vdash, \dashv \in T - A$ - specjalne symbole taśmy - lewego i prawego końca słowa.

ϵ - znak pusty - oznacza brak zapisu na taśmie w danym kroku.

$\sigma = T - \{\vdash, \dashv\} \cup \{\epsilon\}$ - alfabet zapisu := znaki jakie $\mathcal{K}$ może zapisywać na taśmie.

$\delta \subseteq (((Q - \{q_{tak}\}) \times T \times Q \times \sigma \times \{\leftarrow, \circlearrowleft, \rightarrow\}) - (\delta_{bad_dir} \cup \delta_{bad_add}))$ - zbiór przejść, gdzie:

$$\delta_{bad_dir} = \{(q, \vdash, q', t, \leftarrow) \mid q, q' \in Q, t \in \sigma\} \cup \{(q, \dashv, q', t, \rightarrow) \mid q, q' \in Q, t \in \sigma\}$$

To zbiór przejść nielegalnych ze względu na przesunięcie głowicy w kierunku poza słowo (bad direction).

$$\delta_{bad_add} = \{(q, \vdash, q', t, d) \mid q, q' \in Q, t \in \sigma - \{\epsilon\}, d \in \{\leftarrow, \circlearrowleft, \rightarrow\}\}$$

To zbiór przejść nielegalnych ze względu na umieszczenie znaku na lewo od $\vdash$ (bad add).

Natomiast **przejście** $(q, a, q', a', d) \in \delta$ **oznacza**:

- Jeśli automat jest w stanie q, a głowicę ma nad literą a:
- Może zmienić swój stan na q',
- wstawić a' bezpośrednio po lewej od a (ϵ - pusty znak = brak wstawienia),
- na koniec przenieść głowicę o jeden znak w kierunku d ($\circlearrowleft$ oznacza brak ruchu).

Zbiór konfiguracji automatów z klawiaturą oznaczamy jako $\mathcal{C} \subseteq T^* \times Q \times T^*$. Elementy krotki to kolejno znaki przed głowicą, stan automatu, oraz ciąg znaków zaczynający się od aktualnego znaku pod głowicą. Wśród konfiguracji definiujemy:

c_0 - konfiguracja początkowa, $c_0 = (\vdash, q_0, w \dashv)$, gdzie w jest słowem wejściowym.

$\mathcal{C}_{tak}$ - zbiór konfiguracji akceptujących, $\mathcal{C}_{tak} = \{(t, q_{tak}, t') \mid t, t' \in T^*\}$

Definicja przejścia pomiędzy konfiguracjami $c \longrightarrow_{\mathcal{K}} c'$:

Jeśli istnieje $(q, a, q', a', \circlearrowleft) \in \delta$ to istnieją przejścia między konfiguracjami:

$$w, q, (av) \longrightarrow_{\mathcal{K}} wa', q', (av)$$

Jeśli istnieje $(q, a, q', a', \leftarrow) \in \delta$ to ....:

$$\text{Dla } a' \neq \epsilon: \quad w, q, (av) \longrightarrow_{\mathcal{K}} w, q', (a'av), \text{ dla } a' = \epsilon: \quad (wa''), q, (av) \longrightarrow_{\mathcal{K}} w, q', (a''av)$$

Jeśli istnieje $(q, a, q', a', \rightarrow) \in \delta$ to ....:

$$w, q, (av) \longrightarrow_{\mathcal{K}} wa'a, q', v$$

Definicję **biegu** między konfiguracjami wprowadzamy, analogicznie jak dla zwykłych automatów jako domknięcie przechodnio-zwrotne relacji przejścia $\longrightarrow_{\mathcal{K}}$, oznaczamy $\longrightarrow^*_{\mathcal{K}}$.

Finalnie oznaczamy **język rozpoznawany przez automat z klawiaturą** $L(\mathcal{K})$ jako:

$$L(\mathcal{K}) = \{w \in A^* \mid \exists_{c \in \mathcal{C}_{tak}} : \text{ istnieje bieg } c_0 = (\vdash, q_0, w \dashv) \longrightarrow^*_{\mathcal{K}} c\}$$

2 Problem stopu dla $\mathcal{K}$:

Teza: Problem stopu $\text{HALT}_{\mathcal{K}}$ definiowany jako: "Czy dla pary $\langle \mathcal{K}, w \rangle$ zachodzi $w \in L(\mathcal{K})$?", **jest wyłącznie częściowo rozstrzygalny**. Aby udowodnić tezę, zdefiniujmy funkcje:

$\mathbf{M}-$ zbiór maszyn Turinga, $\mathbf{K}-$ zbiór automatów z klawiaturą.

$$f : \mathbf{M} \to \mathbf{K}, \quad f(\mathcal{M}) = \mathcal{K} : L(\mathcal{K}) = L(\mathcal{M})$$
$$f_R : \mathbf{K} \to \mathbf{M}, \quad f_R(\mathcal{K}) = \mathcal{M} : L(\mathcal{M}) = L(\mathcal{K})$$

Lemat: Funkcje f, f_R są obliczalne i dobrze zdefiniowane dla całej dziedziny.
Udowodnijmy, pokazując konstrukcje pozwalające na przekształcenia między automatami dwóch kategorii:

f - $\mathcal{M} \rightsquigarrow \mathcal{K}$: Zdefiniujmy uproszczone automaty z dwoma stosami: $\mathcal{P} = (A, Q, S, q_0, q_F, B, \delta)$.
Funkcję przejść definiujemy przez:

$$\delta \subseteq Q \times (A \cup \{\epsilon\}) \times S^2 \times Q \times (S_{op})^2, \text{ gdzie: } \quad S_{op} = (\{\text{pop}, \text{nop}\} \cup \{\text{push}(a) \mid a \in S\})$$

Gdzie S_{op} to dozwolone operacje stosowe. Akceptacja zachodzi przez stan q_F, a B jest sztucznym symbolem spodu stosu, na którym pop jest zabroniony. Na podstawie dowodu z ćwiczeń (o symulacji taśmy za pomocą dwóch stosów) wiemy, że dla każdego $\mathcal{M} \in \mathbf{M}$ istnieje równoważny $\mathcal{P}$, który jesteśmy w stanie skonstruować wg "recepty". Skąd $\exists$ obliczalna funkcja $f_P : \mathbf{M} \to \mathbf{P}, \quad f_P(\mathcal{M}) = \mathcal{P} : L(\mathcal{M}) = L(\mathcal{P})$ - definiujemy jej wynik jako wynik całkowitej maszyny, która przepisuje słowo kodujące $\mathcal{M}$ zgodnie z tymi zasadami.

Pokażmy symulację $\mathcal{P} \rightsquigarrow \mathcal{K}$ - ustalamy $\mathcal{P} = (A_{\mathcal{P}}, Q_{\mathcal{P}}, S_{\mathcal{P}}, q_{0_{\mathcal{P}}}, q_{F_{\mathcal{P}}}, B, \delta_{\mathcal{P}})$. Ideą konstrukcji będzie pokazanie jak $\mathcal{K}$ może wykorzystywać taśmę do przechowywania stanu stosów $\mathcal{P}$, na lewo od słowa wejściowego. W tym celu wprowadzamy symbole $T' = \{b, s_1, s_2, x, p\}$ oznaczające kolejno:

b = spód stosu, odpowiednik B.
s_1 = na lewo, ciągiem do symbolu s_2, znajduje się reprezentacja pierwszego stosu
s_2 = na lewo, ciągiem do symbolu $\vdash$, znajduje się reprezentacja drugiego stosu
x = ignoruj znak bezpośrednio po lewej
p = znak powrotu, "marker"

Na koniec $\mathcal{K} = (A_{\mathcal{P}}, Q, q_s, q_f, (S_{\mathcal{P}} \cup T'), \sigma, \delta)$. Jego stany to: $Q = (Q_{\mathcal{P}} \times A \times S^2 \times S^2_{op} \times \{\circlearrowleft, \rightarrow\} \times Q_{\mathcal{K}})$, gdzie $Q_{\mathcal{K}}$ to stany logiczne naszego procesu symulacji opisywane w algorytmie poniżej.

Adnotacja: Stany specjalnie rozpisane są mocno nadmiarowo. W praktyce każdy stan logiczny $Q_{\mathcal{K}}$ korzysta tylko z paru elementów krotki, jednak ponieważ korzystają z różnych jej elementów, bardziej precyzyjny zapis byłby dużo mniej zrozumiały i bardzo uciążliwy. Podana krotka oczywiście równie poprawnie spełnia wymóg skończoności.

Nasza maszyna będzie wykonywać **następujący algorytm** w biegu na słowie w:

1. Jeśli stan startowy automatu $\mathcal{P}$ jest stanem akceptującym, akceptuj słowo.
2. Przekształć konfigurację początkową $(\vdash, q_s, w \dashv)$ w $(\vdash bs_1bs_2, (q_{0_{\mathcal{P}}} \times \text{ zaznacz pozycję}), w \dashv)$
3. W pętli powtarzaj:
   - **Zaznacz aktualną pozycję** poprzedzając ją markerem p, zapisz ją w stanie.
   - **Odszukaj symbol na szczycie pierwszego stosu** - pierwszy nie zamazany symbolem x symbol na lewo od s_1, przejdź do stanu szukania symbolu na drugim szczycie
   - Analogicznie znajdź i zapisz **symbol na szczycie drugiego stosu**.

– **Wybierz ruch do realizacji** - Najważniejszy krok w symulacji. Przechodzimy do logicznego etapu "realizacji"ruchu, parametryzowanej efektami przejścia w $\mathcal{P}$ - zmianami stosów $s, s' \in S_{op}^2$, ruchem w prawo do kolejnej litery - $\rightarrow$, lub $\epsilon-$przejściem $= \circlearrowright$, nowym stanem q'. Przy tym samym przejście do takiego nowego parametryzowanego stanu jest możliwe wtedy i tylko wtedy, gdy **istnieje taka kombinacja w relacji δ automatu $\mathcal{P}$.**

– **Realizuj ruch** - Utrzymujemy niezmiennik poprawnego stanu stosów, co robimy poprzez:

pop : Znajdujemy element z góry stosu jak w poprzednich krokach, przesuwamy głowicę na prawo od niego, a następnie wciskamy przed niego znak x, efektywnie go kasując.

nop : Nie musimy nic robić ze stosem, więc możemy przejść do dalszego etapu.

push(a) : Wstawiamy bezpośrednio przed znakiem początku stosu odpowiedni symbol.

Gdzie z osobna realizujemy operację na drugim stosie, a następnie na pierwszym (wracając w prawo).

– **Realizacja c.d.** - Po modyfikacji stosu wracamy do aktualnej pozycji, czyli do pierwszego symbolu od lewej, różnego od x, który jest bezpośrednim następcą znaku p.

– **Porządkuj** - Zmazujemy symbol p za pomocą x, presuwamy się ewentualnie do nast. litery, zmieniamy nasz stan na parę odpowiadającą logicznemu etapowi "zaznacz pozycję"oraz stanowi q' symulowanego automatu $\mathcal{P}$.

– **Akceptuj** - Jeśli stan q' miałby być akceptującym stanem $q_{F_\mathcal{P}}$, kończymy bieg trafiając do stanu akceptującego: $q' \times$ zaznacz pozycję.

Realizacja każdego z etapów algorytmu jest bardzo prosta - wymaga tylko pewnej liczby wewnętrznych stanów logicznych by móc znaleźć specjalny symbol, przeskoczyć nad symbolem po x, zrobić dokładnie jeden krok w prawo, etc. Zatem algorytm będzie bez problemu możliwy do zrealizowania w modelu obliczeń automatów z klawiaturą.

Zauważmy, że maszyna $\mathcal{K}$ działająca zgodnie z powyższym algorytmem zachowuje ścisłą odpowiedniość między pojedynczym krokiem automatu $\mathcal{P}$, a jednym pełnym obrotem głównej pętli symulacji. Pozwoli nam to zdefiniować następujący **niezmiennik, zachowywany na początku obrotu pętli**:

Konfiguracja symulatora $\mathcal{K}$ po wykonaniu pętli n razy jest w poprawnej relacji z pewną konfiguracją $c_\mathcal{P}^n$ automatu $\mathcal{P}$ osiągalną po n krokach z konfiguracji startowej, co oznacza, że:

– Zawartość pierwszego stosu w $c_\mathcal{P}^n$ (czytana od dna) jest dokładnie równa $eval(v_1)$, gdzie v_1 to słowo na taśmie $\mathcal{K}$ pomiędzy markerami s_2 i s_1.

– Zawartość drugiego stosu w $c_\mathcal{P}^n$ jest dokładnie równa $eval(v_2)$, gdzie v_2 to słowo pomiędzy $\vdash$ i s_2.

– Aktualny stan automatu $\mathcal{P}$ zapisany w wielowymiarowej krotce stanu Q maszyny $\mathcal{K}$ jest identyczny ze stanem w konfiguracji $c_\mathcal{P}^n$.

– Słowo w' znajdujące się na taśmie między s_1 a $\dashv$ spełnia $eval(w') = w$, a głowica znajduje się logicznie nad tą samą literą co w konfiguracji $c_\mathcal{P}^n$.

Gdzie $eval(w)$ to funkcja usuwająca wszystkie zmazane symbole z w wraz ze znakami zamazania x.

Powyższy niezmiennik pozwala nam przez obustronne implikacje pokazać równość $L(\mathcal{K}) = L(\mathcal{P})$:

$\Longrightarrow$ **$(L(\mathcal{K}) \subseteq L(\mathcal{P}))$:** Jeśli automat $\mathcal{K}$ zaakceptuje słowo wejściowe w, oznacza to, że istnieje skończony bieg symulacji kończący się w stanie akceptującym. Z niezmiennika pętli wynika, że z tego biegu można odtworzyć poprawny ciąg przejść automatu $\mathcal{P}$ na słowie w, również kończący się w stanie akceptującym. Zatem $w \in L(\mathcal{P})$.

$\Longleftarrow$ **$(L(\mathcal{P}) \subseteq L(\mathcal{K}))$:** Jeśli $w \in L(\mathcal{P})$, to istnieje akceptujący bieg automatu ze stosem. Ponieważ nasza maszyna $\mathcal{K}$ w bloku "Wybierz ruch do realizacji"jest w stanie wybrać i w pełni zasymulować dowolne dozwolone przejście z relacji δ automatu $\mathcal{P}$, zachowując przy tym niezmiennik, to $\mathcal{K}$ jest w stanie odtworzyć krok po kroku ten sam bieg akceptujący, skąd $w \in L(\mathcal{K})$.

Tym samym pokazaliśmy, że $L(\mathcal{P}) = L(\mathcal{K})$. Możemy zatem sposób przekształcenia kodu takiego automatu $\mathcal{P}$ zakodować w pewnej całkowitej maszynie Turinga, tak by otrzymywać odpowiadającą mu maszynę $\mathcal{K}$.

W takim razie na podstawie działania tej maszyny możemy zdefiniować funkcję $f_\mathcal{K}$, przekształcającą $\mathbf{P} \rightarrow \mathbf{K}$.

Ponieważ złożenie dwóch funkcji obliczalnych jest obliczalne, tym samym udowodniliśmy też możliwość skonstruowania obliczalnej i dobrze zdefiniowanej dla całej dziedziny funkcji $f = f_\mathcal{K} \circ f_\mathcal{P}$, c.n.d.

f_R - $\mathcal{K} \rightsquigarrow \mathcal{M}$ - Zauważmy, że jedyna operacja dostępna automatom z klawiaturą, której maszyny Turinga nie są w stanie zasymulować wprost, to wstawienie z rozepchnięciem. Zatem aby skonstruować maszynę Turinga odpowiadającą automatowi $\mathcal{K}$ wystarczy wszystkie operacje wykonywać analogicznie, a jedynie w miejsce wstawienia znaku wywoływać następujący algorytm:

0. Niech nasza maszyna B.S.O. ma do dyspozycji drugą taśmę, ze startu z samymi blankami.
1. Zapamiętujemy wewnątrz naszego stanu: stan $\mathcal{K}$ w którym mamy być po przejściu, relatywną pozycję względem wpisanego symbolu, na której mielibyśmy się znaleźć (0, 1 lub 2 miejsca na prawo) oraz literę którą mieliśmy wstawić.
2. Kopiujemy literę spod głowicy na drugą taśmę, w jej miejsce wstawiamy marker.
3. Wszystkie symbole na prawo od markera wrzucamy na drugą taśmę po naszej literze.
4. Odnajdujemy pozycję markera, na prawo od niej przepisujemy całą drugą taśmę.
5. Czyścimy drugą taśmę.
6. Wracamy do markera, zastępujemy go zapamiętaną literą.
7. Przesuwamy się o tyle miejsc w prawo ile zapamiętaliśmy.

Co zachowuje niezmiennik, zgodnie z którym maszyna $\mathcal{M}$ ma po każdym przejściu, z ewentualną operacją wstawienia z rozsuwaniem, stan odpowiadający stanowi symulowanej $\mathcal{K}$, jak i pozycję głowicy oraz identyczne słowo między blankami, jakie w $\mathcal{K}$ występuje między $\vdash$ i $\dashv$.

Dodanie takich przejść oczywiście wymaga jedynie skończonej liczby stanów rzędu $|Q| \times |A| \times 3$, więc jest akceptowalne. Analogicznie do podpunktu wyżej jesteśmy zatem w stanie zakodować całkowitą maszynę Turinga tak, aby przekształcała $\mathcal{K}$ na wejściu w $\mathcal{M}$ taką że $L(\mathcal{K}) = L(\mathcal{M})$. Czym samym pokazaliśmy, że istnieje taka dobrze zdefiniowana, obliczalna funkcja f_R.

Korzystając z tych funkcji i ich własności pokażmy, że problem $\text{HALT}_\mathcal{K}$ jest równie trudny co HALT dla maszyn Turinga, zdefiniowany analogicznie. Korzystamy z obustronnej redukcji:

$\text{HALT}_\mathcal{M} \leq \text{HALT}_\mathcal{K}$ Zauważmy, że dla pary wejściowej $\langle u, w\rangle$, gdzie u = kod maszyny Turinga, w = słowo, $(u, w) \in L(\text{HALT}_\mathcal{M})$ na mocy lematu zachodzi wtw. gdy $(f(u), w) \in L(\text{HALT}_\mathcal{K})$, jako że $f(u)$ rozpoznaje identyczny język. Ponadto skoro f jest obliczalna, to funkcja g definiowana przez $g(\langle u, w\rangle) = \langle f(u), w\rangle$,, też oczywiście jest obliczalna. Zatem pokazaliśmy, że $\text{HALT}_\mathcal{M}$ redukuje się do $\text{HALT}_\mathcal{K}$.

$\text{HALT}_\mathcal{K} \leq \text{HALT}_\mathcal{M}$. Analogicznie tylko zamiast z g korzystamy z $g_R : g_R(\langle u, w\rangle) = \langle f_R(u), w\rangle$.

A zatem skoro problemy te są równej trudności, a ponadto wiemy, że $\text{HALT}_\mathcal{M} \in RE \setminus DEC$, to tym samym udowodniliśmy również naszą tezę, zgodnie z którą $\text{HALT}_\mathcal{K} \in RE \setminus DEC$, czyli **nie jest rozstrzygalny**, ale jest częściowo rozstrzygalny.